Projet facultatif

Démineur

Corrigé

EPHEC — Bachelier en informatique, niveau 2

Année académique 2026-2027

Comment lire ce corrigé

C'est une façon d'écrire chaque méthode, pas la seule. Si vos onze tests sont au vert avec un code différent, votre code est juste. Comparez tout de même : la question n'est pas « est-ce que ça marche ? », les tests y ont répondu, mais « est-ce que ça se lit ? ». Un cas traité en moins, une condition plus simple, un nom plus clair : c'est ce que le corrigé peut vous apprendre.

Les réponses aux « Questions à se poser » et à « Pour aller plus loin » suivent chaque étape. Elles ne sont pas testées : relisez-les avec Grille ouverte à côté.

Les étapes

D.1 Lire ce qui est fourni

Rien à écrire. Les deux questions, elles, méritent une réponse.

Pourquoi deux constructeurs ? Que deviendraient les tests si les mines étaient toujours placées au hasard ?

Un test doit savoir d'avance ce qu'il attend. `minesVoisines` affirme qu'il y a une mine voisine en (1,1) parce que le test a posé lui-même la mine en (0,0). Avec un placement au hasard, le test ne saurait ni où sont les mines, ni combien en touchent une case : il ne pourrait rien affirmer, ou seulement des choses vagues, « le compte est entre 0 et 8 ». Le constructeur à deux arguments crée une grille vide, et `poserMine` laisse le test décider. C'est la même idée que de passer un `Random` en paramètre plutôt que d'en créer un dans la classe : ce qui est aléatoire se confie à l'appelant, pour que le test puisse le remplacer.

Le constructeur refuse une grille de neuf mines sur neuf cases. Que se passerait-il sans ce contrôle ? Lisez placerAuHasard avant de répondre.

`placerAuHasard` tire des positions au hasard jusqu'à avoir posé le nombre de mines demandé, et ne compte une mine que si la case était libre. Avec neuf mines sur neuf cases, la neuvième finit par se poser ; mais avec dix, ou plus, la boucle cherche une case libre qui n'existe pas, et ne s'arrête jamais. Le programme se fige, sans message, sans exception : rien ne dit ce qui se passe. Le contrôle du constructeur transforme ce silence en `IllegalArgumentException` qui nomme le problème. Et neuf mines sur neuf cases, même si la boucle s'en sortait, donnerait une partie perdue avant le premier clic : il reste au moins une case saine, c'est la règle.

D.2 Compter les mines voisines

```
public int minesVoisines(int ligne, int colonne) {
    int compte = 0;
    for (int dl = -1; dl <= 1; dl++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dl == 0 && dc == 0) {
                continue;
            }
            int l = ligne + dl;
            int c = colonne + dc;
            if (dansLaGrille(l, c) && mines[l][c]) {
                compte++;
            }
        }
    }
    return compte;
}
```

Les deux boucles parcourent les neuf décalages, `continue` écarte la case elle-même, et `dansLaGrille` protège l'accès au tableau : sans lui, une case du bord ferait lire `mines[-1][...]` et lèverait une `ArrayIndexOutOfBoundsException`. Écrire les huit voisines une à une marche aussi ; c'est huit fois la même condition, et une faute de frappe s'y cache mieux que dans deux boucles.



D.3 et D.4 Découvrir une case, puis propager

Les deux étapes ne font qu'une méthode. Le bloc final est la propagation de l'étape D.4 ; tout ce qui le précède est l'étape D.3.

```
public void decouvrir(int ligne, int colonne) {
    if (!dansLaGrille(ligne, colonne) || perdue) {
        return;
    }
    if (etats[ligne][colonne] != Etat.CACHEE
            && etats[ligne][colonne] != Etat.DOUTE) {
        return;
    }

    etats[ligne][colonne] = Etat.DECOUVERTE;

    if (mines[ligne][colonne]) {
        perdue = true;
        return;
    }

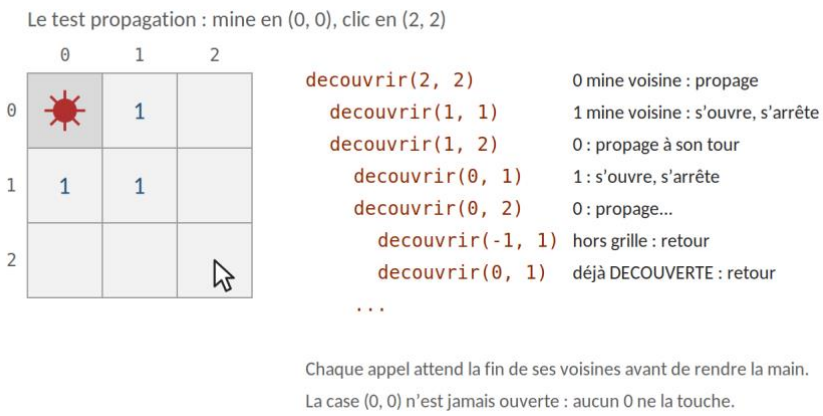
    if (minesVoisines(ligne, colonne) == 0) {
        for (int dl = -1; dl <= 1; dl++) {
            for (int dc = -1; dc <= 1; dc++) {
                decouvrir(ligne + dl, colonne + dc);
            }
        }
    }
}
```

L'ordre des contrôles compte. Le premier `if` protège les tableaux, comme en D.2, et il permet à la propagation d'appeler `decouvrir` sur les neuf décalages sans se demander lesquels existent. La case elle-même est dans les neuf : elle est déjà DECOUVERTE à ce moment-là, et le second `if` la rejette aussitôt. Deux conditions de moins dans la boucle, et aucun cas particulier.

Le second `if` ne laisse passer que CACHEE et DOUTE : il bloque donc DECOUVERTE et DRAPEAU en une seule condition. On peut l'écrire en nommant les deux états bloqués ; c'est équivalent, mais si un cinquième état apparaît un jour, il faut savoir quelle version choisir : celle qui dit ce qui passe, ou celle qui dit ce qui est bloqué.

Pourquoi la récursion s'arrête-t-elle ? Quelle ligne de l'étape D.3 l'empêche de tourner en rond entre deux cases vides voisines ?

La case passe à DECOUVERTE *avant* que ses voisines soient appelées. Quand une voisine vide propage à son tour et revient sur la première case, celle-ci n'est plus CACHEE : le second `if` la renvoie sans rien faire. Chaque case n'est donc découverte qu'une fois, et comme la grille est finie, les appels finissent. Inversez les deux lignes, en appelant les voisines avant de marquer la case, et deux cases vides voisines s'appellent l'une l'autre sans fin, jusqu'à une `StackOverflowError`.



La trace des premiers appels sur le test propagation ; chaque niveau d'indentation est un appel de plus sur la pile.

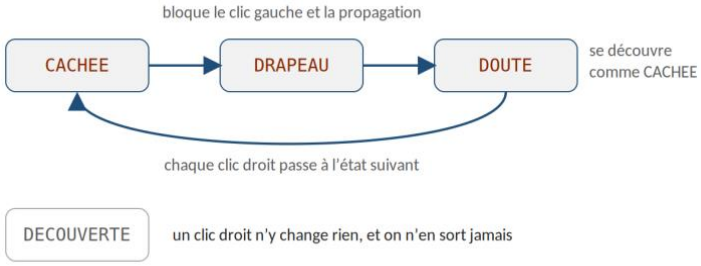
La propagation découvre-t-elle une case marquée d'un drapeau ? Une case en doute ? Est-ce le comportement que vous attendiez en jouant ?

Un drapeau n'est pas découvert : la propagation passe par `decouvrir`, et le drapeau la bloque comme il bloque un clic. Un doute, lui, se découvre. Le drapeau reste donc planté au milieu d'une zone ouverte, s'il a été posé au mauvais endroit : c'est ce que fait le jeu d'origine, et c'est cohérent avec la règle. Un drapeau est une décision du joueur, et le programme ne revient pas sur une décision à sa place. Si vous trouvez cela gênant en jouant, la règle à changer est dans Grille, pas dans la vue.

D.5 Marquer une case

```
public void basculerDrapeau(int ligne, int colonne) {
    switch (etats[ligne][colonne]) {
        case CACHEE -> etats[ligne][colonne] = Etat.DRAPEAU;
        case DRAPEAU -> etats[ligne][colonne] = Etat.DOUTE;
        case DOUTE -> etats[ligne][colonne] = Etat.CACHEE;
        case DECOUVERTE -> { }
    }
}
```

Le `switch` sur une énumération a un avantage sur une suite de `if` : le compilateur sait quels états existent. Le cas DECOUVERTE est écrit même s'il ne fait rien, pour dire que ce n'est pas un oubli. Une version avec un tableau d'états suivants, ou une méthode `suivant()` dans l'énumération, est tout aussi juste et plus courte ; ici, l'enchaînement tient en trois lignes qui se lisent sans aide.



D.6 Gagner, perdre, terminer

```
public boolean estGagnee() {
    if (perdue) {
        return false;
    }
    int decouvertes = 0;
    for (int l = 0; l < lignes; l++) {
        for (int c = 0; c < colonnes; c++) {
            if (etats[l][c] == Etat.DECOUVERTE && !mines[l][c]) {
                decouvertes++;
            }
        }
    }
    return decouvertes == casesSaines;
}

public boolean estTerminee() {
    return perdue || estGagnee();
}
```

Le test `&& !mines[l][c]` a l'air superflu : une case minée découverte, c'est une partie perdue, et la méthode a déjà rendu `false`. Il protège pourtant contre une évolution : si un jour la partie perdue montre toutes les mines en les passant à DECOUVERTE, le compte resterait juste. Une autre voie est de tenir un compteur de cases découvertes dans `decouvrir` et de comparer deux entiers : plus rapide, mais un compteur de plus à garder juste. Sur 81 cases, parcourir la grille ne coûte rien.

Pour aller plus loin

Le premier clic. Dans la plupart des démineurs, le premier clic ne tombe jamais sur une mine. Ici, il peut. Que faudrait-il changer pour l'empêcher ? Dans Grille, dans la vue, ou dans les deux ? Pourquoi le constructeur actuel ne le permet-il pas ?

Le constructeur place les mines avant que quiconque ait cliqué : il ne peut pas éviter une case qu'il ne connaît pas encore. La règle « le premier clic est toujours sain » est une règle du jeu, donc une affaire de Grille : on retarde le placement jusqu'au premier appel de `decouvrir`. Le constructeur retient le nombre de mines sans les poser ; un champ `minesPlacees` dit si c'est fait ; et `decouvrir` commence par poser les mines, en écartant la case cliquée, la première fois qu'on l'appelle. La vue n'a rien à savoir : elle appelle `decouvrir` comme avant. Il reste un point à trancher : le constructeur de test, qui pose les mines à la main, doit continuer à fonctionner sans attendre un clic. Le contrôle du constructeur, lui, suffit tel quel : avec au plus une mine de moins que de cases, il reste toujours une case pour le clic.

Les drapeaux restants. Le jeu d'origine affiche le nombre de mines moins le nombre de drapeaux posés. Quelle méthode faudrait-il ajouter à Grille, et que devrait faire la vue de sa réponse ?

Une méthode `int drapeauxRestants()` dans Grille : le nombre de mines moins le nombre de cases à l'état DRAPEAU, compté en parcourant `etats`. Le nombre de mines n'est pas gardé aujourd'hui : on l'obtient par `lignes * colonnes - casesSaines`, ou en ajoutant un champ. Le résultat peut être négatif si le joueur pose plus de drapeaux qu'il n'y a de mines ; le jeu d'origine l'affiche tel quel, et c'est un signal utile. La vue ajoute un `Label` et le met à jour dans `redessiner`, qui redessine déjà tout à partir du modèle. Elle n'a rien à compter : elle affiche ce qu'on lui rend. Et la méthode se teste sans fenêtre, comme les autres.

Une grille plus grande. Essayez une grille de cent lignes sur cent colonnes, presque sans mines. Que devient la récursion de la propagation ? Comment pourrait-on s'en passer ?

Chaque appel de `decouvrir` qui propage attend la fin de ses huit appels avant de rendre la main : les appels s'empilent. Sur une grille presque vide, la propagation peut traverser des milliers de cases d'une traite, et la pile de la machine virtuelle est limitée. Le programme s'arrête alors sur une `StackOverflowError`. À quelle taille exactement, cela dépend de la machine et des options de lancement : ce qui est sûr, c'est que ça finit par casser, et qu'en 9 × 9 on en est très loin.

Pour s'en passer, on remplace la pile d'appels par une collection que l'on tient soi-même : une file de cases à visiter. On y met la case cliquée ; tant qu'elle n'est pas vide, on en retire une case, on la découvre, et si elle n'a aucune mine voisine, on y ajoute ses voisines encore cachées. Une `ArrayDeque<int[]>` ou une liste convient. Le même parcours, sans empilement d'appels, et la profondeur ne dépend plus de la taille de la zone. C'est un exercice de plus : les onze tests de GrilleTest doivent rester au vert, et ils ne disent rien de la façon dont la propagation est écrite. C'est leur rôle.